Regression Testing Framework

High-Level Design Document — Terminus Data Platform

Version: 2.0 | **Date:** April 2026 | **Status:** Draft for Review

1. Executive Summary

Field	Details
Project Name	Terminus Data Platform - Regression Testing Framework
Objective	Automated detection and validation of schema changes in BigQuery tables managed via Terraform
Scope	GCP-based data platform with BigQuery, GCS, and Terraform-managed infrastructure
Approach	Baseline comparison model integrated directly into GCP CI/CD
Target Environment	UET (User Environment Testing) — dedicated regression environment
Estimated Duration	4 weeks (2 phases)

Design Principles:

- **Fully GCP-Native** — All data, triggers, and execution within GCP
- **Minimal Infrastructure** — Single Cloud Run Job handles all operations
- **Self-Contained** — Input data generated and managed within GCP
- **Cost Optimized** — Pay only when tests run (~$60/month)

2. Problem Statement

The Terminus data platform operates at scale with hundreds of BigQuery tables managed via Terraform. Key challenges:

Pain Point	Impact	Severity
Schema changes via Terraform not tracked	Silent downstream breakages	**High**
No fixed-input test harness	Cannot verify output consistency	**Medium**
No dedicated regression environment	CI/CD cannot simulate real BigQuery dependencies	**Medium**
Manual validation effort	Hours of engineer time per release	**High**

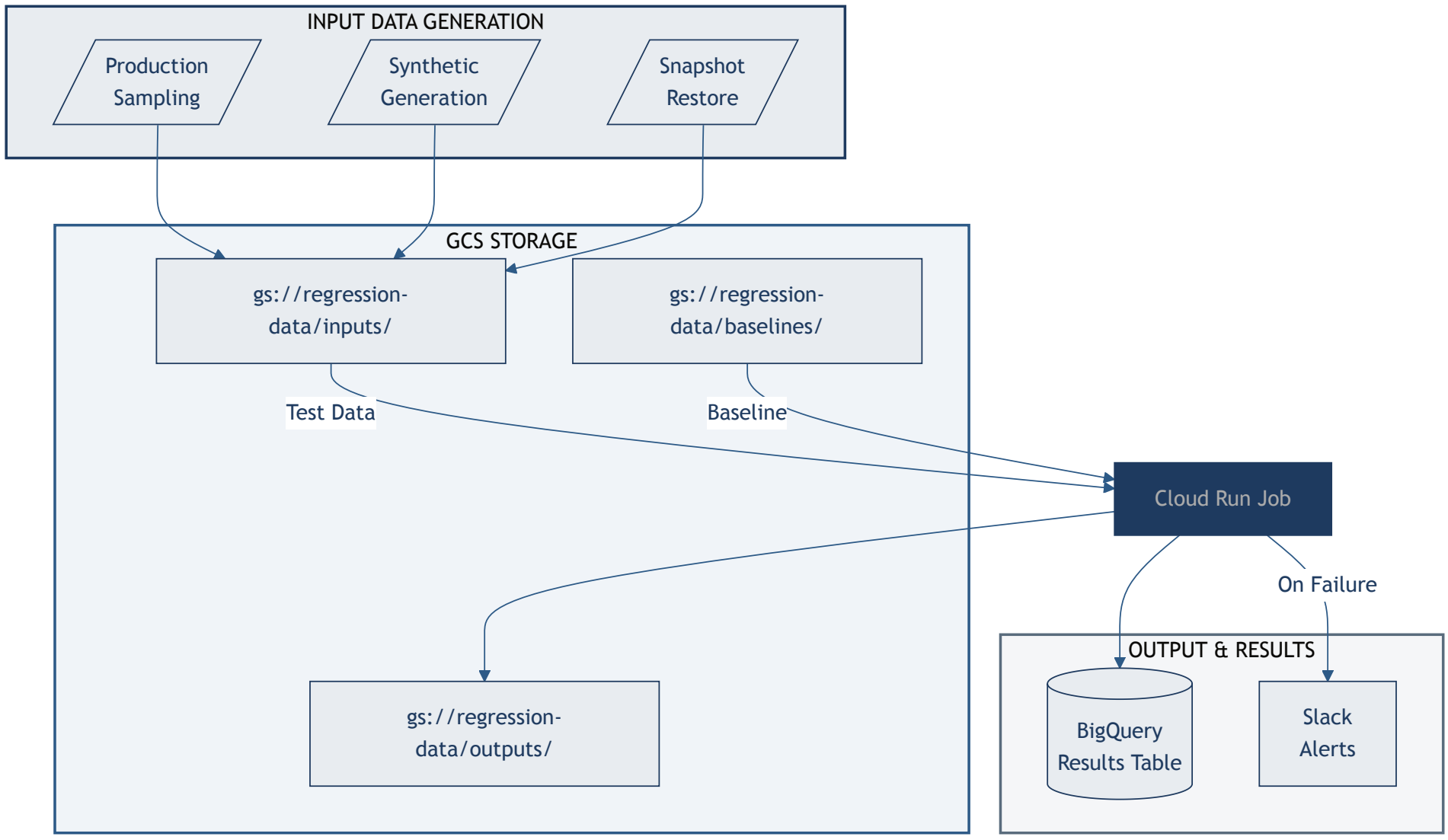
3. Solution Overview

A fully GCP-native regression testing framework that:

- Captures baseline schema snapshots from BigQuery `INFORMATION_SCHEMA`
- Generates test input data dynamically from production samples or BigQuery scripts
- Stores all data in GCS/BigQuery — no external storage dependencies
- Triggers via Cloud Build or Eventarc when Terraform state changes
- Reports results to BigQuery and Slack

Input/Output Data Flow

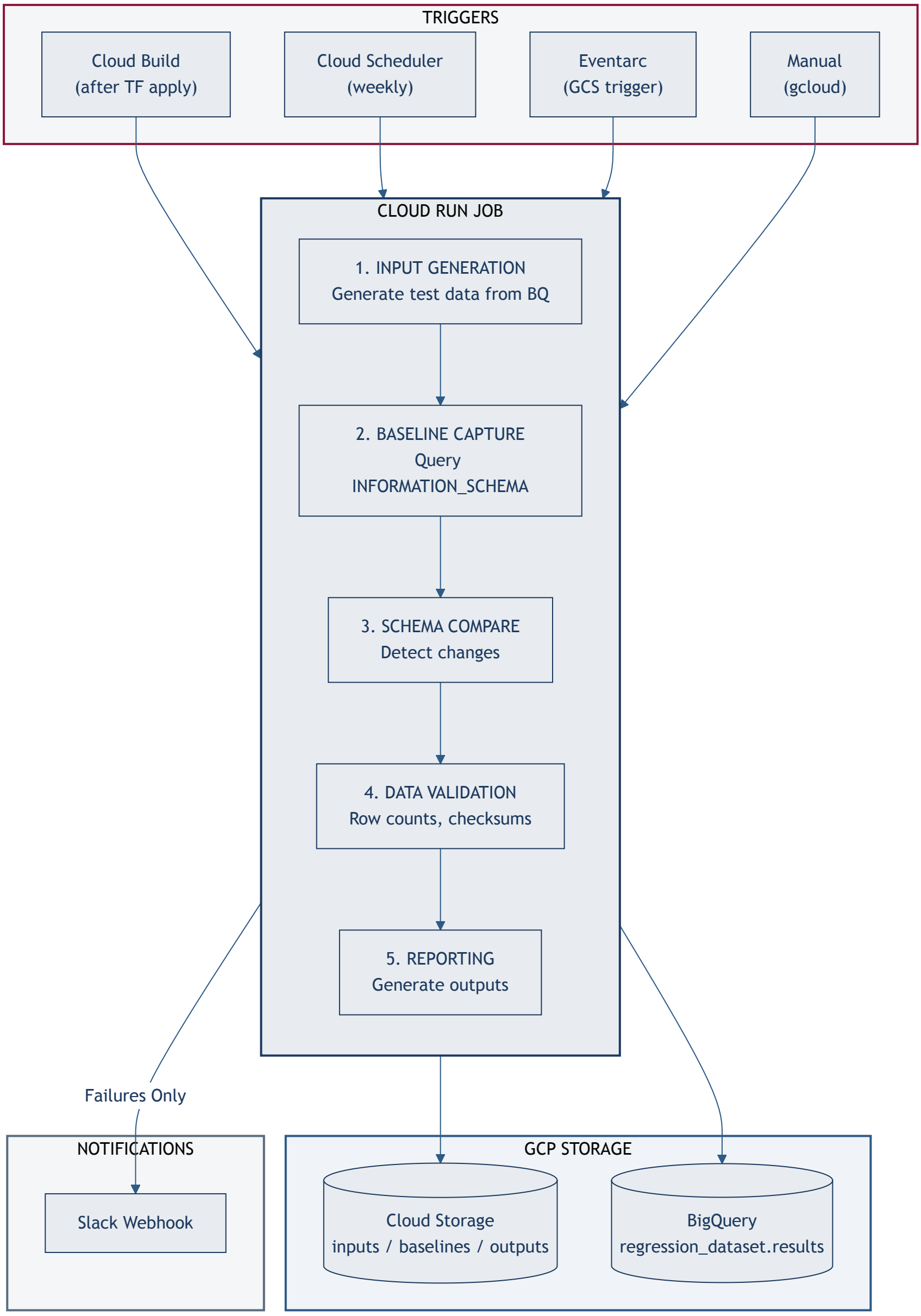
Figure 3.1: Data Generation & Storage Flow



4. Architecture

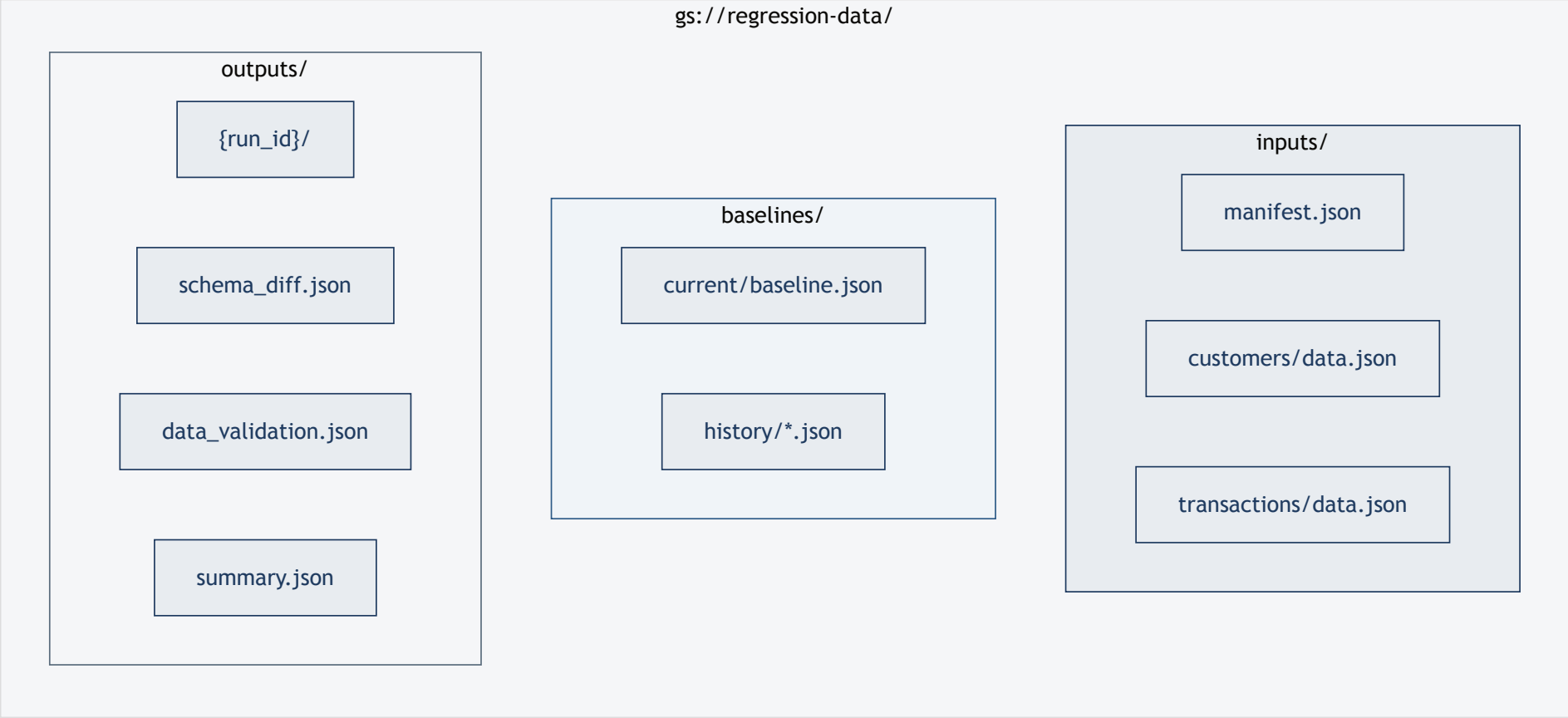
4.1 High-Level Architecture (GCP-Native)

Figure 4.1: System Architecture Overview



4.2 GCS Data Organization

Figure 4.2: Storage Bucket Structure

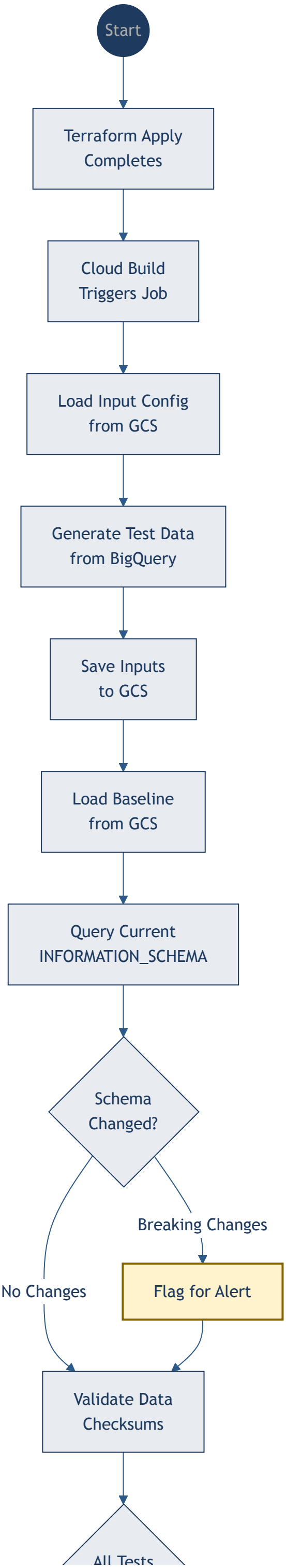


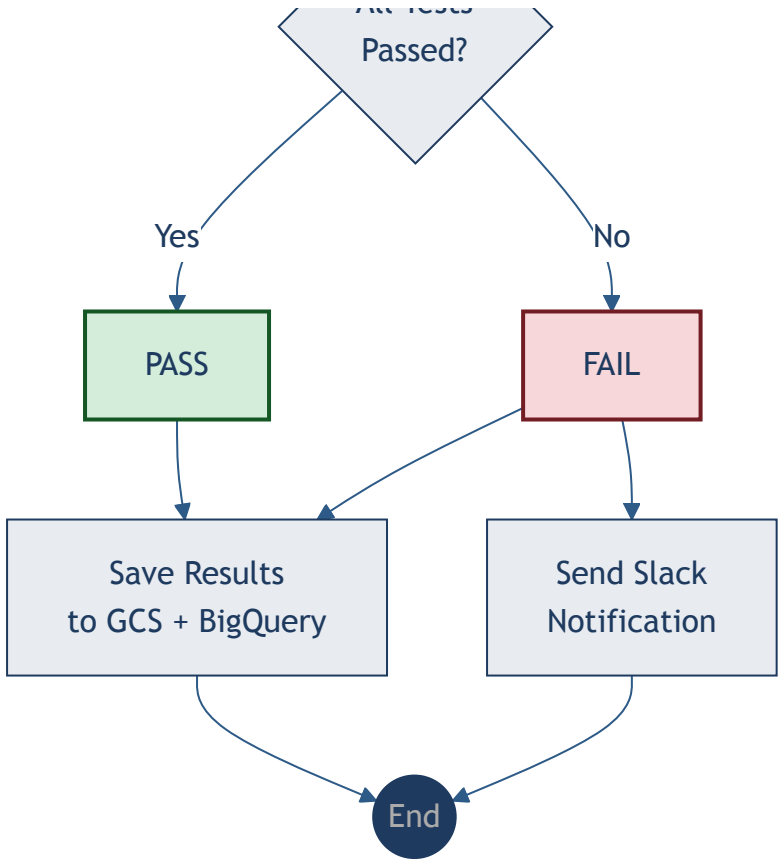
4.3 Trigger Options

Trigger	Use Case	Configuration	Priority
Cloud Build	Run after `terraform apply`	Add step in cloudbuild.yaml	**Primary**
Cloud Scheduler	Weekly baseline refresh	Cron: `0 2 * * 0`	**Baseline**
Eventarc	When TF state changes	Monitor GCS bucket	**Optional**
Manual	On-demand testing	`gcloud run jobs execute`	**Debug**

4.4 Complete Execution Flow

Figure 4.3: End-to-End Regression Test Flow


Start
Terraform Apply Completes
Cloud Build Triggers Job
Load Input Config from GCS
Generate Test Data from BigQuery
Save Inputs to GCS
Load Baseline from GCS
Query Current INFORMATION_SCHEMA
Schema Changed?
Breaking Changes
No Changes
Flag for Alert
Validate Data Checksums



5. Technology Stack

Layer	Technology	Rationale
Compute	Cloud Run Job	Pay-per-use, up to 1hr execution, no infra management
Triggers	Cloud Build / Scheduler / Eventarc	GCP-native, no external CI dependency
Storage	GCS + BigQuery	Native GCP, cost-effective, queryable results
Language	Python 3.11+	google-cloud SDK, robust BigQuery support
Notifications	Slack Webhook	Simple integration, no additional setup
IaC	Terraform	Consistent with existing infrastructure

Key Benefits:

- **Fully GCP-Native** — No GitHub/GitLab dependency for execution
- **Cloud Build integration** — Triggers after `terraform apply`
- **Self-contained data management** — Inputs generated from BigQuery, outputs stored in GCS
- **Minimal maintenance** — Single Cloud Run Job, automated triggers

6. Component Specifications

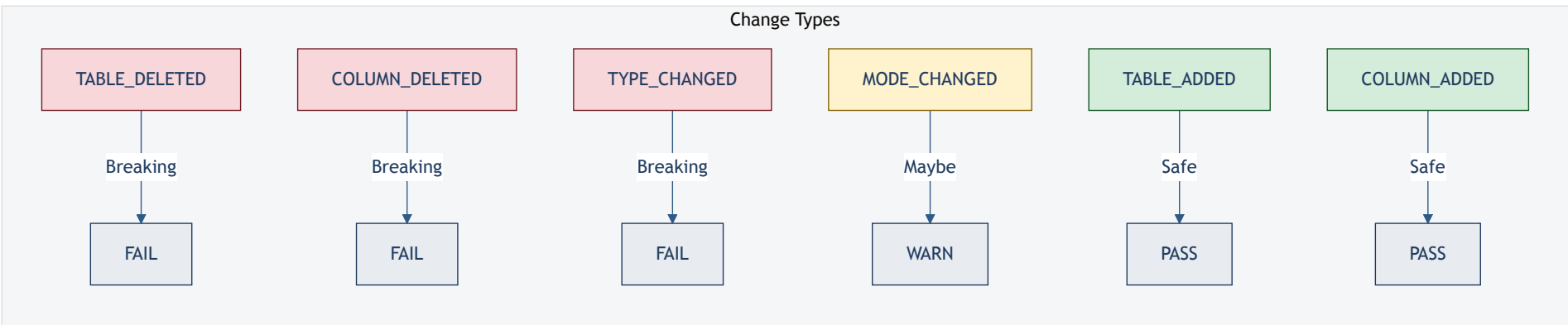
6.1 Input Data Generator

Generates test input data dynamically within GCP — no external file uploads needed.

Strategy	Use Case	Implementation
Production Sampling	Realistic data	BQ query with `LIMIT` + `WHERE` filters
Synthetic Generation	Controlled edge cases	BQ scripted INSERT with `GENERATE_ARRAY`
Snapshot Restore	Point-in-time data	BQ snapshot table or GCS export

6.2 Schema Comparator

Figure 6.1: Schema Change Detection



6.3 Data Validator

Validates data integrity using efficient BigQuery operations:

Validation Type	PASS Condition	FAIL Condition
Hash-Based	`current_hash == baseline_hash`	Mismatch detected
Row Count	`current_count == baseline_count`	Count differs
Aggregates	All aggregate values match	Value differs

7. Implementation Plan

Phase 1: Core Framework

Week 1–2

Task	Owner	Deliverable
Create GCS bucket	Infrastructure	`gs://regression-data/`
Create BigQuery dataset	Infrastructure	`regression_dataset.results`
Implement input generator	Backend	`InputDataGenerator` class
Implement baseline manager	Backend	`capture_baseline()`
Set up Cloud Run Job	Infrastructure	Job deployable via Terraform
Configure Cloud Build trigger	Infrastructure	Post-terraform-apply trigger

Phase 2: Validation & Alerts

Week 3–4

Task	Owner	Deliverable
Implement output manager	Backend	`OutputManager` class
Implement data validator	Backend	`validate_data()`
Configure Cloud Scheduler	Infrastructure	Weekly baseline refresh
Slack integration	DevOps	Failure alerts working
End-to-end testing	QA	Full workflow validated
Documentation	All	Runbook + README

> **Total Duration: 4 weeks** — Reduced from 8 weeks in original design

8. Cost Estimation

Monthly Cost Breakdown

Component	Est. Cost	Notes
Cloud Run Job	$10 – $25	~100 invocations/month, 2 vCPU, 2GB RAM
GCS Storage	$2 – $8	Inputs + outputs + baselines (~5 GB)
BigQuery	$15 – $40	Query costs for input generation + results
Cloud Scheduler	$1 – $3	Weekly triggers
Cloud Build	$0 – $5	First 120 min/day free

Cost Comparison

Original Design

$185

/month

4 Cloud Functions, GitHub Actions, SendGrid, Looker Studio

RECOMMENDED

GCP-Native

$60

/month

Cloud Run Job, Cloud Build, Cloud Scheduler

Hybrid (GitHub)

$50

/month

GitHub Actions + Cloud Run (external dependency)

9. Success Criteria

100%	<5%	<10m	<2m	100%	<2h
Schema Changes Detected	False Positive Rate	Execution Time	Alert Latency	Table Coverage	Monthly Maintenance

10. Risks & Mitigations

Risk	Probability	Impact	Mitigation
Baseline drift — baseline becomes stale	Medium	High	Auto-refresh baseline weekly via Cloud Scheduler
Large table checksums slow	Medium	Medium	Use row count only for tables > 10M rows
Terraform state not synced	Low	Medium	Cross-validate with live INFORMATION_SCHEMA
Input data becomes stale	Medium	Medium	Regenerate inputs weekly; use date-relative filters

11. Appendix

A. Cloud Build Configuration

```yaml
# cloudbuild.yaml - Integrated with Terraform deployment
steps:
  # Step 1: Terraform Init & Plan
  - name: 'hashicorp/terraform:1.7'
    entrypoint: 'sh'
    args:
      - '-c'
      - |
        cd terraform/
        terraform init -backend-config="bucket=${_TF_STATE_BUCKET}"
        terraform plan -json -out=plan.tfplan > /workspace/plan_output.json

  # Step 2: Terraform Apply
  - name: 'hashicorp/terraform:1.7'
    id: 'terraform-apply'
    entrypoint: 'sh'
    args:
      - '-c'
      - |
        cd terraform/
        terraform apply -auto-approve plan.tfplan

  # Step 3: Run Regression Tests (after apply)
  - name: 'gcr.io/google.com/cloudsdktool/cloud-sdk'
    entrypoint: 'gcloud'
    args:
      - 'run'
      - 'jobs'
      - 'execute'
      - 'regression-tester'
      - '--region=us-central1'
      - '--wait'
      - '--args=--mode=full'
    waitFor: ['terraform-apply']
```

B. Input Configuration Example

```json
{
  "version": "1.0",
  "tables": [
    {
      "name": "customers",
      "source_table": "production_dataset.customers",
      "strategy": "sample",
      "sample_size": 500,
      "filter": "created_date >= DATE_SUB(CURRENT_DATE(), INTERVAL 90 DAY)"
    },
    {
      "name": "transactions",
      "source_table": "production_dataset.transactions",
      "strategy": "sample",
      "sample_size": 2000
    },
    {
      "name": "edge_cases",
      "strategy": "synthetic",
      "row_count": 100
    }
  ],
  "refresh_schedule": "weekly"
}
```

C. Glossary

Term	Definition
UET	User Environment Testing — dedicated non-production GCP project
Baseline	Versioned snapshot of schemas and data checksums
Breaking Change	Schema change that may break existing consumers
Checksum	FARM_FINGERPRINT hash of table data for integrity validation
Cloud Build	GCP CI/CD service that runs build steps in containers
Eventarc	GCP event routing service that triggers Cloud Run on GCS/Pub/Sub events